

INDEPENDENT TECHNICAL REVIEW

PyJWT 2.13.0 - Five Security-Behavior Claims

Same-object independent review and reviewer-comparison determination

Review ID	ITR-PYJWT-2130-INDEPENDENT-v1.0
Exact object	PyJWT 2.13.0 installed distribution; compared with PyJWT 2.12.1 vulnerable baseline
Stated source commit	7144e4534c34810f4525dc4578a32add8212cff
Claims reviewed	C001-C005 from the PyJWT 2.13.0 security release notes
Independent environment	CPython runtime in isolated target directories
Independent test posture	Separately authored differential behavior harness
Comparison target	Jake Macdonald, ITR-001 PyJWT 2.13.0 review
Review date	20 July 2026

OBJECT RESULT	PASS - FIVE RELEASE CLAIMS HOLD IN THE INDEPENDENT TEST BOUNDARY
Basis	Every reviewed fixed behavior passed on PyJWT 2.13.0. The same independently authored tests failed on PyJWT 2.12.1, providing fail-before/pass-after discrimination.

REVIEWER RESULT	PARTIAL QUALIFICATION - BOUNDED RELEASE-CLAIM VERIFICATION
Basis	Jake reached the same five technical outcomes and preserved environment limits. His review is substantively correct on the selected claims. Qualification should remain narrow because his counterexamples are mostly direct confirmations and his public package does not itself provide the differential harness used in this independent review.

1. Executive Answer

This review corrects the prior object mismatch. Jake Macdonald reviewed PyJWT 2.13.0 against five security claims. This independent review examined the same PyJWT release and the same five claims, then compared the two reviews to determine what Jake's work demonstrates about his reviewing ability.

The independent technical result is clear: all five fixed behaviors hold in PyJWT 2.13.0 under the tested CPython lane. The same independent harness fails against PyJWT 2.12.1 on every claim, including both subconditions of C005. This differential is important because it shows that the tests distinguish the vulnerable baseline from the fixed release rather than merely producing expected PASS text.

Jake's five PASS classifications are therefore technically supported within the same narrow claim boundary. He did not miss a contrary result on the five selected claims. The meaningful difference is review depth and proof construction: our independent review adds vulnerable-version differential testing, multiple serialization and scheme variants, and a directly runnable test carrier.

2. Exact Claims Reviewed

Claim	Release behavior
C001	Reject JWK JSON documents passed as raw HMAC secrets.
C002	Bind the token header algorithm to PyJWK.algorithm_name during verification.
C003	Reject non-http(s) URI schemes in PyJWKClient.
C004	Preserve the cached JWK Set when a fetch error occurs.
C005	For b64=false detached JWS, require an empty compact payload segment and require b64 in crit.

3. Independent Method

- Installed PyJWT 2.13.0 into an isolated target directory.
- Installed PyJWT 2.12.1 into a separate isolated target directory as the pre-fix baseline.
- Authored an independent Python harness without copying Jake's exact test file.
- Executed the same harness separately against each version.
- Required the fixed release to reject or preserve behavior as stated.
- Required the older release to demonstrate the pre-fix behavior, thereby creating a fail-before/pass-after differential.
- Captured machine-readable JSON results identifying the imported module path, package version, test name, outcome, and observed exception or state.

The review does not claim a full PyJWT security audit. It evaluates only the five release-note claims selected by Jake and adds a differential baseline sufficient to test whether those fixes are behaviorally observable.

4. Independent Results

Claim/test	2.13.0	2.12.1 baseline	Differential	Independent conclusion
C001 raw JWK JSON rejected as HMAC secret	PASS	FAIL	YES	PASS
C002 JWT alg must match PyJWK.algorithm_name	PASS	FAIL	YES	PASS
C003 PyJWKClient rejects non-http(s) URI schemes	PASS	FAIL	YES	PASS

C004 cached JWK set preserved after fetch error	PASS	FAIL	YES	PASS
C005a b64=false rejects nonempty compact payload segment	PASS	FAIL	YES	PASS
C005b b64=false requires crit declaration	PASS	FAIL	YES	PASS

4.1 Observed behavior

C001 raw JWK JSON rejected as HMAC secret: rejected 3 JWK JSON variants; ordinary secret accepted

C002 JWT alg must match PyJWK.algorithm_name: InvalidAlgorithmError: Token algorithm 'HS256' does not match the key's algorithm 'HS512'

C003 PyJWKClient rejects non-http(s) URI schemes: rejected 6 schemes; accepted http/https

C004 cached JWK set preserved after fetch error: cached JWK set preserved after URLError

C005a b64=false rejects nonempty compact payload segment: DecodeError: Payload segment must be empty when 'b64' is false.

C005b b64=false requires crit declaration: InvalidTokenError: The 'b64' header parameter requires 'b64' to be listed in 'crit'.

4.2 Vulnerable-baseline discrimination

PyJWT 2.12.1 failed every independent fixed-behavior test: raw JWK JSON was not rejected as required; the PyJWK algorithm mismatch did not raise the fixed-release algorithm-binding error; non-http(s) URI construction was accepted; fetch failure cleared the cache; nonempty b64=false compact payloads were not rejected; and b64=false without crit was accepted. This establishes direct behavioral separation between the two releases.

5. Comparison With Jake Macdonald's Review

Dimension	Jake's review	Independent review	Contrast	Reviewer judgment
Object and claim boundary	PyJWT 2.13.0; five release-note security claims.	Same object and same five claims.	Aligned.	PASS
Technical outcomes	All five claims PASS under CPython 3.12.	All five fixed behaviors PASS on 2.13.0.	Same result.	PASS
Baseline discrimination	No exposed fail-before/pass-after run in the submitted qualification carrier.	Same harness run against 2.12.1; all fixed-behavior tests fail.	Independent review proves the tests discriminate the fixed release.	PARTIAL
C001 depth	One raw JWK JSON example plus ordinary secret control.	Three serialization/key-type variants plus ordinary secret control.	Independent review is broader, but Jake identified the correct property.	PASS / narrow
C002 depth	One HS256 token vs HS512 PyJWK mismatch.	Same core mismatch behavior independently derived.	No material disagreement.	PASS
C003 depth	Five rejected URI forms and HTTP/HTTPS controls.	Six rejected schemes and HTTP/HTTPS controls.	Similar; both remain scheme-level, not full SSRF destination review.	PASS / bounded
C004 depth	One URLError with populated cache.	One URLError with explicit cache-state assertion.	Substantively equivalent.	PASS
C005 depth	Two tests: nonempty payload segment and missing crit.	Two independently derived tests, including a 4096-byte middle segment.	Same technical outcome; neither is a full resource-exhaustion benchmark.	PASS / bounded
Unknown preservation	Full interpreter matrix and lint held when unavailable.	Review claims only the executed isolated CPython lane.	Jake's limitation discipline is appropriate.	PASS
Conclusion control	Five-claim PASS; broader security claims excluded.	Five-claim PASS; broader security claims excluded.	Aligned and proportionate on the technical object.	PASS
Public reproducibility carrier	Narrative, tests, saved outputs, and hashes	One runnable harness plus JSON output for both fixed	Jake's carrier is less direct and lacks differential	PARTIAL



6. Reviewer Qualification Determination

Judging Jake by contrast against an independent review of the same object produces a materially different and more accurate conclusion than auditing only his repository presentation.

6.1 Competence demonstrated

- He selected a genuine third-party artifact and fixed the release and commit.
- He extracted the correct five release claims.
- He located relevant implementation behavior and upstream tests.
- He authored tests that reached the correct behavioral results.
- His five PASS classifications agree with the independent same-object review.
- He preserved missing environment coverage as HOLD and did not promote the result into a complete security assurance claim.

6.2 Limits demonstrated

- His reviewer-authored tests are primarily targeted release-claim confirmations rather than broad vulnerability discovery.
- His submitted public carrier does not include a simple differential harness that proves fail-before/pass-after behavior.
- His term 'counterexamples' is stronger than the actual test posture for several claims.
- The review demonstrates competence in bounded Python source-code and release-claim verification, not general qualification across contracts, formal mathematics, hardware, or consequence-boundary systems.

6.3 Final reviewer outcome

QUALIFICATION	PARTIAL QUALIFICATION - BOUNDED PYTHON RELEASE-CLAIM REVIEW
Basis	Jake's technical conclusions on the same object are corroborated by an independent differential review. He demonstrates real bounded competence. Full QUALIFIED FOR BOUNDED REVIEW should require one further submission showing deeper independently originated challenge or vulnerability discovery, or a clean verifier that another party can execute without reconstructing his evidence package.

7. Claim-by-Claim Reviewer Comparison

C001 - Reject JWK JSON documents passed as raw HMAC secrets.

Jake reached the correct result. The independent review expanded the property across three JWK JSON variants and confirmed ordinary-secret compatibility. No substantive error found; depth was narrower.

C002 - Bind the token header algorithm to PyJWK.algorithm_name during verification.

Jake reached the correct algorithm-binding result. The independent run observed InvalidAlgorithmError on 2.13.0 and a different pre-fix outcome on 2.12.1. No substantive error found.

C003 - Reject non-http(s) URI schemes in PyJWKClient.

Jake correctly verified scheme allowlisting. Both reviews remain bounded to URI schemes and do not establish comprehensive SSRF destination controls. His PASS is valid only for the release claim as written.

C004 - Preserve the cached JWK Set when a fetch error occurs.

Jake correctly verified that a populated JWK Set cache survives a simulated URL fetch failure. The independent review reproduced the same state property and confirmed that 2.12.1 clears the cache.

C005 - For b64=false detached JWS, require an empty compact payload segment and require b64 in crit.

Jake correctly separated the two required b64=false behaviors. The independent differential confirmed both fixes. His use of a large compact segment is a functional guard test, not a performance or DoS-amplification measurement; the claim should remain behavioral.

8. Evidence Package and Reproduction Files

File	Purpose	SHA-256
independent_pyjwt_2_13_0_review_harness.py	Exact independently executed test harness	02c17c0590c0a848e1e79f8d04409c8687a7900a56b2 aae483cbd3970c9ff824
independent_results_pyjwt_2.13.0.json	Machine-readable PyJWT 2.13.0 results	2892c50c3abdf2379fe861fc88a0dc53ec09762a754a1 4b9e1d39d681b91f9bd
independent_results_pyjwt_2.12.1.json	Machine-readable PyJWT 2.12.1 baseline results	8c8effa686a40b8d8d4adbd9fc918337370bc75558b a08f18e595227e07b070

9. Source Basis

- PyJWT 2.13.0 changelog at commit 7144e4534c34810f4525dc4578a32addd8212cff.
- PyJWT official GitHub security-advisory index for the five 2.13.0 security fixes.
- Jake Macdonald's ITR-001 principal review and reviewer-authored test file.
- Independent isolated installations of PyJWT 2.13.0 and PyJWT 2.12.1.
- Independent test harness and captured JSON results included with this report.

10. Declaration and Limits

This report records an independently executed same-object behavioral review. It does not certify PyJWT, does not establish that no other vulnerabilities exist, and does not qualify Jake Macdonald outside the bounded artifact class and scope stated above. The comparison judges the reviewer by agreement, disagreement, omissions, challenge depth, uncertainty handling, and proportionality against an independently produced review of the same object.